



Универзитет „Св. Кирил и Методиј“ во Скопје
**ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И
КОМПЈУТЕРСКО ИНЖЕНЕРСТВО**

Факултет за информатички науки и компјутерско инженерство

Универзитет Св. Кирил и Методиј — Скопје

Континуирана интеграција и испорака

CalorieChecker

Систем за проверка на калории од храна со AI

Студент: Виктор Никифоровски

Проф. Др. Панче Рибарски

Асс. Маг. Стефан Андонов

Асс. Димитар Милески

28.06.2026

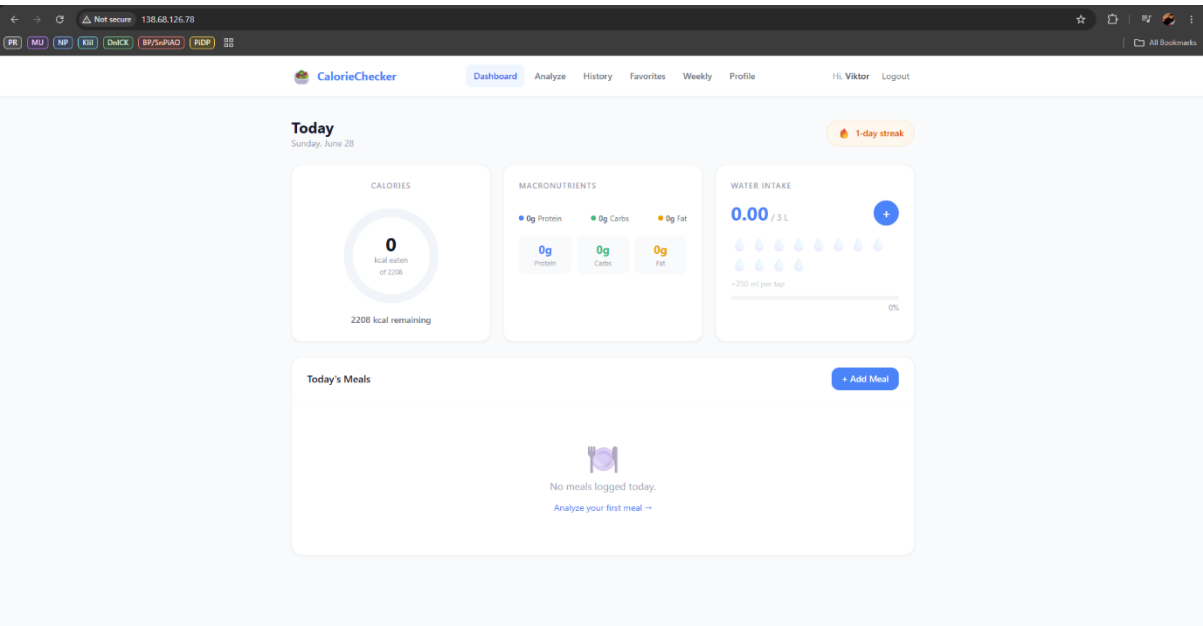
1. Опис на проектот

CalorieChecker е веб апликација за проверка на калории од храна со AI. Корисникот внесува опис на оброк или прикачува слика, а апликацијата преку Anthropic Claude API враќа проценка на калории и макронутриенти. Идејата за проектот е од предметот Дизајн на интеракцијата човек-компјутер, но проектот е изработен за предметот Континуирана интеграција и испорака, со фокус на контејнеризација, CI/CD автоматизација и Kubernetes деплојмент.

GitHub репозиториум: <https://github.com/vnikiforovski/CalorieChecker>

DockerHub: <https://hub.docker.com/u/vnikiforovski0604>

Апликацијата е деплојувана и достапна на: <http://138.68.126.78>



2. Технологии

Апликацијата е изградена со следниот tech stack:

Слој	Технологија
Backend	Django + Django REST Framework, Gunicorn (3 workers)
Frontend	React (Vite) + Tailwind CSS, Nginx
База на податоци	PostgreSQL 15
AI	Anthropic Claude API (claude-sonnet-4-6)
Контејнеризација	Docker (multi-stage builds), Docker Compose
CI/CD	GitHub Actions, GitFlow workflow
Kubernetes	DigitalOcean DOKS, kubectl, Helm
Безбедност	Trivy (Docker image scanning), JWT, NetworkPolicies

3. Контејнеризација

3.1 Докеризација на апликацијата

Секој сервис има свој Dockerfile со multi-stage build. Кај backend-от, Stage 1 (builder) инсталира Python dependencies, а Stage 2 ги копира само инсталираните пакети без build алатки. Кај frontend-от, Stage 1 го build-ува React со Vite, а Stage 2 го сервира преку Nginx без Node.js.

entrypoint.sh автоматски извршува django migrate при секое стартување на контејнерот, пред да го стартува Gunicorn со 3 workers.

backend/Dockerfile

```
FROM python:3.11-slim AS builder

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim

WORKDIR /app

COPY --from=builder /install /usr/local

COPY . .

EXPOSE 8000

COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
CMD ["/entrypoint.sh"]
```

frontend/Dockerfile

```
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

3.2 Docker Compose оркестрација

docker-compose.yml оркестрира три сервиси: db (PostgreSQL 15), backend (Django/Gunicorn преку backend/Dockerfile) и frontend (React/Nginx преку

frontend/Dockerfile). Дефинирани се две Docker мрежи — frontend-network и backend-network — со цел frontend да не може директно да пристапи до базата. PostgreSQL податоците се зачувуваат во persistent volume. Редоследот на стартување е контролиран преку depends_on — backend чека db да биде healthy, а frontend чека backend да стартува, со цел да се избегнат грешки при поврзување во моментот на иницијализација.

docker-compose.yml

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5434:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - backend-network
    deploy:
      resources:
        limits:
          memory: 512M
          cpus: '0.5'

  backend:
    build: ./backend
    env_file: .env
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - media_files:/app/media
    ports:
      - "8000:8000"
    healthcheck:
      test: ["CMD-SHELL", "curl -f http://localhost:8000/api/health/ || exit 1"]
      interval: 30s
      timeout: 10s
      retries: 3
    networks:
      - backend-network
      - frontend-network
    deploy:
      resources:
        limits:
          memory: 512M
          cpus: '0.5'

  frontend:
    build: ./frontend
    depends_on:
```

```

    - backend
  ports:
    - "80:80"
  networks:
    - frontend-network
  deploy:
    resources:
      limits:
        memory: 256M
        cpus: '0.25'

volumes:
  postgres_data:
  media_files:

networks:
  backend-network:
  frontend-network:

```

4. CI/CD Pipeline — GitHub Actions

Проектот користи GitFlow workflow со пет типа на branches: *feature/*, *develop*, *release/*, *hotfix/* и *master*. GitHub Actions workflows се дефинирани за — *feature/* тригерира lint проверка, *develop* тригерира целосен CI pipeline, а *master*, *release/* и *hotfix/* тригерираат CD pipeline со деплојмент.

4.1 GitFlow и Workflow структура

Branch	Workflow	Jobs
feature/*	feature.yml	Lint (flake8)
develop	ci.yml	Lint → Test → Docker Build → Trivy Security Scan
master /; release/*; hotfix/*; v*.*	cd.yml	Test → DockerHub Push → K8s Deploy → GitHub Release

4.2 Feature Branch Pipeline (feature.yml)

При секој push на *feature/** branch се извршува само flake8 lint проверка на Python кодот. Целта е брз feedback за ~30 секунди — без тестови и без Docker build. Feature branches се краткотрајни и не се деплојуваат, па целосната CI проверка чека до merge во *develop*.

4.3 CI Pipeline (ci.yml)

При секој push на *develop* branch се извршуваат четири последователни jobs:

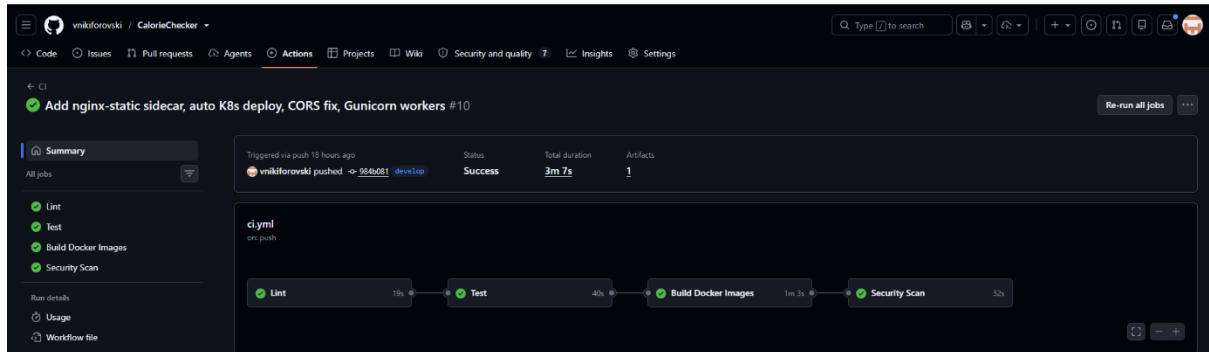
Lint: flake8 проверка на Python код и Vite build проверка на frontend.

Test: Django тестови се извршуваат против вистински PostgreSQL service container. Тестирани се health check endpoint, автентикација и регистрација.

Build: Docker images за backend и frontend се build-уваат за да се верификува дека Dockerfiles се исправни. Images не се push-уваат на DockerHub.

Security Scan: Trivy го скенира backend Docker image за познати CVE пропусти (CRITICAL и HIGH severity). Резултатот се зачувува во GitHub Security tab.

GitHub Actions — CI pipeline на develop branch



4.4 CD Pipeline (cd.yml)

При push на master, release/* branch или git tag (v*..) се активира целосниот CD pipeline:

Test: Истите Django тестови повторно се извршуваат — никогаш не се push-ува broken image.

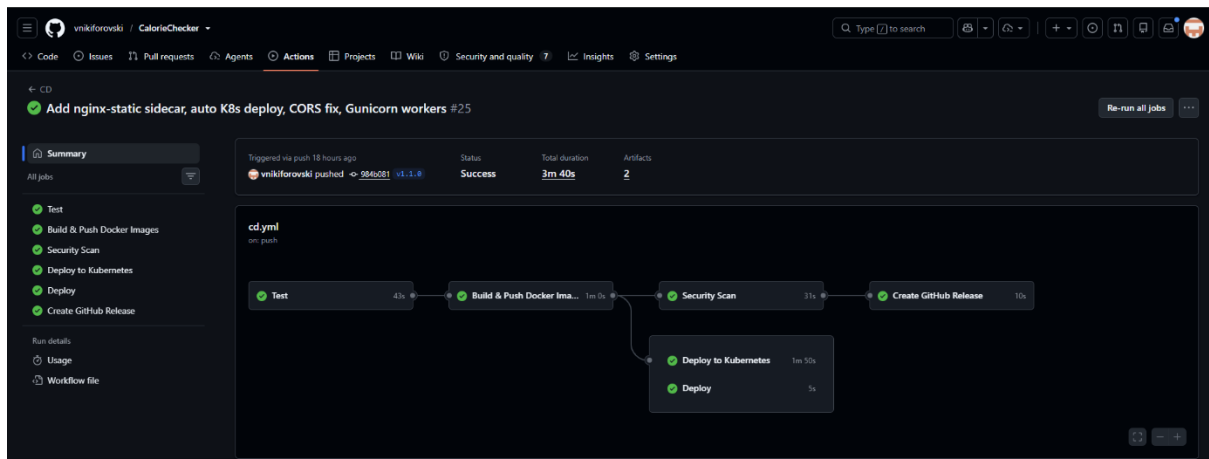
Build & Push: Docker images се build-уваат и push-уваат на DockerHub со три тага: :latest, :<commit-sha7> и :<v1.x.x> (само на tag push). Commit hash тагот овозможува rollback до точна верзија.

Security Scan: Trivy скенира во SARIF формат и резултатот се качува во GitHub Security Code Scanning.

Deploy to Kubernetes: doctl се автентичира со DigitalOcean и автоматски извршува kubectl rollout restart на backend и frontend deployments. Kubernetes повлекува нови images и прави rolling update без downtime.

GitHub Release: При git tag push се создава автоматски GitHub Release со changelog од commit пораките и Docker pull командите за новата верзија.

GitHub Actions — CD pipeline



DockerHub — images со latest, sha и version тарови

vnikiiforovski0604/caloriechecker-frontend

By vnikiiforovski0604 · Updated about 19 hours ago

Overview Tags

Sort by Newest Filter tags

TAG	Digest	OS/ARCH	Last pull	Compressed size
964b881	3f18152b9ec3	linux/amd64	less than 1 day	24.98 MB
latest	3f18152b9ec3	linux/amd64	less than 1 day	24.98 MB
v1.1.0	e1e4a2237d6c	linux/amd64	less than 1 day	24.98 MB

vnikiiforovski0604/caloriechecker-backend

By vnikiiforovski0604 · Updated about 19 hours ago

Overview Tags

Sort by Newest Filter tags

TAG	Digest	OS/ARCH	Last pull	Compressed size
964b881	152674ce5328	linux/amd64	less than 1 day	75.29 MB
latest	152674ce5328	linux/amd64	less than 1 day	75.29 MB
v1.1.0	0a8eb13b0242	linux/amd64	less than 1 day	75.29 MB

5. Kubernetes манифести

Апликацијата е деплојувана на DigitalOcean Kubernetes кластер (DOKS) во Frankfurt со 2 nodes (s-2vcpu-4gb, Kubernetes v1.36.0). Сите манифести се во k8s/ папката и се применети во namespace caloriechecker-prod.

5.1 Deployment, ConfigMaps и Secrets

Backend Deployment користи 2 replicas со Rolling Update стратегија (maxSurge: 1, maxUnavailable: 0) за zero-downtime deployments. Сензитивните податоци (DJANGO_SECRET_KEY, POSTGRES_PASSWORD, ANTHROPIC_API_KEY) се чуваат во Kubernetes Secret (base64 encoded). Несензитивната конфигурација (POSTGRES_HOST, DEBUG, POSTGRES_DB) е во ConfigMap.

Backend pod-от содржи повеќе containers: два init containers (wait-for-postgres и collect-static) и nginx-static sidecar кој го сервира /static/ на port 8080 преку shared emptyDir volume.

Pod Anti-Affinity осигурува дека двата backend pods се распоредени на различни nodes. Security Context поставува runAsNonRoot: true и allowPrivilegeEscalation: false.

Frontend Deployment исто така користи 2 replicas со Rolling Update и Pod Anti-Affinity. Nginx го сервира React SPA на port 80 со try_files поддршка за React Router. За разлика од backend, frontend pod-от нема init containers, secrets или environment variables — React апликацијата е pre-compiled во статични фајлови при Docker build и не бара runtime конфигурација.

kubectl get pods -n caloriechecker-prod -o wide — pods на различни nodes

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
backend-c55658f77-8khkr	2/2	Running	0	21h	10.108.0.84	caloriechecker-pool-3caecy	<none>	<none>
backend-c55658f77-9jng9	2/2	Running	0	21h	10.108.0.160	caloriechecker-pool-3caecf	<none>	<none>
django-migrate-k8bgg	0/1	Completed	1 (24h ago)	24h	10.108.0.190	caloriechecker-pool-3caecf	<none>	<none>
frontend-5579fc4c57-8khlg	1/1	Running	0	21h	10.108.0.2	caloriechecker-pool-3caecy	<none>	<none>
frontend-5579fc4c57-8twx4	1/1	Running	0	21h	10.108.0.237	caloriechecker-pool-3caecf	<none>	<none>
postgres-0	1/1	Running	0	24h	10.108.0.135	caloriechecker-pool-3caecf	<none>	<none>

5.2 Service

Сите три сервиси (backend, frontend, postgres) користат ClusterIP тип — достапни се само внатре во кластерот. Backend сервисот изложува два порти: 8000 за Django API и 8080 за nginx-static sidecar. Надворешниот пристап е овозможен само преку Ingress Controller.

kubectl get services -n caloriechecker-prod

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
backend	ClusterIP	10.109.18.168	<none>	8000/TCP, 8080/TCP	24h
frontend	ClusterIP	10.109.22.231	<none>	80/TCP	24h
postgres	ClusterIP	10.109.17.160	<none>	5432/TCP	24h

5.3 Ingress

Nginx Ingress Controller е инсталиран преку Helm и создава DigitalOcean LoadBalancer со јавна IP адреса (138.68.126.78).

Ingress дефинира четири routing правила:

- /api/ → backend Service:8000 (Django REST API)
- /admin/ → backend Service:8000 (Django Admin панел)
- /static/ → backend Service:8080 (nginx-static sidecar)
- / → frontend Service:80 (React SPA)

kubectl get ingress -n caloriechecker-prod — LoadBalancer IP

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
caloriechecker-ingress	nginx	*	138.68.126.78	80	24h

5.4 StatefulSet за базата

PostgreSQL е деплојуван како StatefulSet (наместо Deployment) бидејќи базата е stateful — чува траен state на диск. StatefulSet обезбедува стабилно DNS ime (postgres-0) и PersistentVolumeClaim (5Gi DigitalOcean block storage) за зачувување на податоците при рестартирање на pod-от.

```
kubectl get statefulset -n caloriechecker-prod
```

NAME	READY	AGE
postgres	1/1	25h

```
kubectl get pvc -n caloriechecker-prod — PersistentVolumeClaim
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
postgres-data	Bound	pvc-ed048314-169e-4c29-aa04-39e2f63137e8	5Gi	RWO	do-block-storage	<unset>	25h
postgres-data-postgres-0	Bound	pvc-f71b3111-7d6e-4c50-96d2-7efe78b3e770	5Gi	RWO	do-block-storage	<unset>	25h

5.5 Namespace, ResourceQuota и LimitRange

Namespace caloriechecker-prod ги содржи сите ресурси на апликацијата. ResourceQuota дефинира максимални ограничувања за целиот namespace: 20 pods, 4 CPU cores и 4Gi RAM. LimitRange поставува default resource requests и limits за секој pod кој не дефинира свои, со цел да се спречи ненамерно трошење на ресурси.

```
kubectl get all -n caloriechecker-prod — преглед на сите ресурси
```

NAME	READY	STATUS	RESTARTS	AGE
pod/backend-c55658f77-8khkr	2/2	Running	0	22h
pod/backend-c55658f77-9jng9	2/2	Running	0	22h
pod/django-migrate-k8bvg	0/1	Completed	1 (24h ago)	24h
pod/frontend-5579fc4c57-8khlg	1/1	Running	0	22h
pod/frontend-5579fc4c57-8twx4	1/1	Running	0	22h
pod/postgres-0	1/1	Running	0	25h

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/backend	ClusterIP	10.109.18.168	<none>	8080/TCP,8080/TCP	24h
service/frontend	ClusterIP	10.109.22.231	<none>	80/TCP	24h
service/postgres	ClusterIP	10.109.17.160	<none>	5432/TCP	25h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/backend	2/2	2	2	24h
deployment.apps/frontend	2/2	2	2	24h

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/backend-56b9c67498	0	0	0	24h
replicaset.apps/backend-5b5584d77f	0	0	0	23h
replicaset.apps/backend-68b8d74cbc	0	0	0	22h
replicaset.apps/backend-68fd59dd99	0	0	0	22h
replicaset.apps/backend-6fcd747d67	0	0	0	24h
replicaset.apps/backend-7f97b5c77d	0	0	0	24h
replicaset.apps/backend-c55658f77	2	2	2	22h
replicaset.apps/frontend-5579fc4c57	2	2	2	22h
replicaset.apps/frontend-5868bb96bc	0	0	0	22h
replicaset.apps/frontend-5b7fbb7c49	0	0	0	24h
replicaset.apps/frontend-5f5f679b4f	0	0	0	24h
replicaset.apps/frontend-689c66d964	0	0	0	23h
replicaset.apps/frontend-76579f9b78	0	0	0	23h
replicaset.apps/frontend-f5884bcc9	0	0	0	24h

NAME	READY	AGE
statefulset.apps/postgres	1/1	25h

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/backend-hpa	Deployment/backend	cpu: <unknown>/70%, memory: <unknown>/80%	2	5	2	24h
horizontalpodautoscaler.autoscaling/frontend-hpa	Deployment/frontend	cpu: <unknown>/70%	2	4	2	24h

NAME	STATUS	COMPLETIONS	DURATION	AGE
job.batch/django-migrate	Complete	1/1	23s	24h

5.6 Дополнителни Kubernetes ресурси

Покрај задолжителните ресурси, имплементирани се и:

HPA (Horizontal Pod Autoscaler): Автоматски скалира backend (2-5 pods) и frontend (2-4 pods) кога CPU надминува 70% или Memory 80%.

PodDisruptionBudget: Осигурува дека минимум 1 pod е секогаш достапен при node maintenance.

NetworkPolicy (Zero-Trust): deny-all policy блокира сиот сообраќај, а специфични allow policies отвораат само потребните патеки: frontend → backend:8000, backend → postgres:5432, backend → интернет (Anthropic API), сите pods → CoreDNS:53.

Kubernetes Job: django-migrate job со init container кој чека PostgreSQL да биде достапен пред да ги изврши database migrations.

6. Заклучок

Проектот CalorieChecker го покрива целосниот DevOps циклус. Апликацијата е контејнеризирана со Docker, оркестрирана со Docker Compose, автоматизирана со GitHub Actions CI/CD pipeline (GitFlow, Trivy security scanning, автоматски GitHub Releases), и деплојувана на production Kubernetes кластер на DigitalOcean.

CD pipeline-от е целосно автоматизиран — секој push на master branch автоматски build-ува нови Docker images, ги push-ува на DockerHub со semantic versioning, и деплојува новата верзија во Kubernetes кластерот без рачна интервенција.